JOBSHEET 5

SORTING (BUBBLE, SELECTION & INSERTION SORT)

Nama : Fadila Mayamasita

NIM : 254107020012

Kelas : 1C_Teknik informatika

D-IV TEKNIK INFORMATIKA

TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

5.2 PRAKTIKUM 1 – Mengimpelentasikan sorting menggnakan sort

5.2.1 Langkah Praktikum 1

A.Sorting Bubble Sort

1. Buat folder baru dengan nama Praktikum05.

2. Buat class Sorting, kemudian tambahkan atribut

Jawaban :

```
public class Sorting {

    int [] data;
    int jumlah;
}
```

3. Buatlah konstruktor dengan parameter Data[] dan jmlData :

Jawaban:

```
Sorting (int Data[], int jmlDat){
    jumData=jmlDat;
    data=new int[jmlDat];
    for (int i=0; i <jumData; i++){
        data[i]= Data[i];
    }
```

4. Buatlah method bubbleSort bertipe void dan deklarasikan isinya menggunakan algoritma Bubble Sort.

Jawaban :

```
void bubbleSort(){
    int temp=0;
   for (int i=0; i<jumData-1; i++){
    for (int j=1; j<jumData-i; j++){
        if(data[j-1]>data[j]){
            temp=data[j];
            data[j]=data[j-1];
            data[j-1]=temp;
        }
    }
   }
}
```

5. Buatlah method tampil bertipe void dan deklarasikan isi method tersebut.

Jawaban :

```
void tampil(){
    for(int i=0; i<jumData; i++){
        System.out.print(data[i]+" ");
    }
    System.out.println(x: );
}
```

6. Buat class SortingMain kemudian deklarasikan array dengan nama a[] kemudian isi array tersebut

Jawaban:

```
public class SortingMain13 {
    int a[]= {20, 10, 2, 7, 12};

}
```

7. Buatlah objek baru dengan nama dataurut1 yang merupakan instansiasi dari class Sorting, kemudian isi parameternya

Jawaban :

```
Sorting dataurut1 = new Sorting (a, a.length);
```

8. Lakukan pemanggilan method bubbleSort dan tampil

Jawaban:

```
System.out.println(x: "Data awal 1");
dataurut1.tampil();
dataurut1.bubbleSort();
System.out.println(x: "Data sudah diurutkan dengan BUBBLE SORT(ASC)");
dataurut1.tampil();
```

9. Jalankan program, dan amati hasilnya!

Jawaban :

```
D semester 2_45a0cd4a\bin' 'MINGGU5.SortingMa
Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT(ASC)
2 7 10 12 20
```

5.2.2 Verifikasi hasil percobaan

B.Sorting selection sort

1. Pada class Sorting yang sudah dibuat di praktikum sebelumnya tambahkan method SelectionSort yang mengimplementasikan pengurutan menggunakan algoritma selection sort.

Jawaban :

```
void SelectionSort(){
    for(int i=0; i<jumData-1; i++){
        int min=i;
        for (int j=i+1; j<jumData; j++){
            if(data[j]<data[min]){
                min=j;
            }
        }
        int temp=data[i];
        data[i]=data[min];
        data[min]=temp;
    }
}
```

2. Deklarasikan array dengan nama b[] pada kelas SortingMain kemudian isi array tersebut

Jawaban :

```
int b[]= {30, 20, 2, 8, 14};
```

3. Buatlah objek baru dengan nama dataurut2 yang merupakan instansiasi dari class Sorting, kemudian isi parameternya

Jawaban :

```
Sorting dataurut2 = new Sorting(b, b.length);
```

4. Lakukan pemanggilan method SelectionSort dan tampil

Jawaban:

```
System.out.println(x: "Data awal 2");
dataurut2.tampil();
dataurut2.SelectionSort();
System.out.println(x: "Data sudah diurutkan dengan SELECTION SORT (ASC)");
dataurut2.tampil();
```

5. Jalankan program dan amati hasilnya!

Jawaban :

```
n' 'MINGGU5.SortingMain13'
Data awal 1
20 10 2 7 12
Data sudah diurutkan dengan BUBBLE SORT(ASC)
2 7 10 12 20
Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan SELECTION SORT (ASC)
2 8 14 20 30
```

5.2.3 Verifikasi Hasil Percobaan

C.Sorting – Insertion sort

1. Pada class Sorting yang sudah dibuat di praktikum sebelumnya tambahkan method insertionSort yang mengimplementasikan pengurutan menggunakan algoritma insertion sort.

Jawaban:

```
void InsertionSort(){
    for (int i=1; i<=data.length-1; i++){
        int temp=data[i];
        int j=i-1;
        while (j>=0 && data[j]>temp){
            data[j+1]=data[j];
            j--;
        }
        data[j+1]=temp;
    }
}
```

2. Deklarasikan array dengan nama c[] pada kelas SortingMain kemudian isi array tersebut

Jawaban:

```
int c[]= {40, 10, 4, 9, 3};
```

3. Buatlah objek baru dengan nama dataurut3 yang merupakan instansiasi dari class Sorting, kemudian isi parameternya

Jawaban:

```
Sorting dataurut3 = new Sorting(c, c.length);
```

4. Lakukan pemanggilan method insertionSort dan tampil

Jawaban:

```
System.out.println(x: "Data Awal 3");
dataurut3.tampil();
dataurut3.InsertionSort();
System.out.println(x: "Data sudah diurutkan dengan Insertion SORT (ASC)");
dataurut3.tampil();
```

5. Jalankan program dan amati hasilnya!

Jawaban:

```
Data Awal 3
40 10 4 9 3
Data sudah diurutkan dengan Insertion SORT (ASC)
3 4 9 10 40
```

5.2.5 Pertanyaan!

1. Jelaskan fungsi kode program berikut

```
if (data[j-1]>data[j]){
    temp=data[j];
    data[j]=data[j-1];
    data[j-1]=temp;
}
```

Jawaban: Menukan posisi dua angka jika angka di sebelah kiri lebih besar dari pada angka di sebelah kanan.

2. Tunjukkan kode program yang merupakan algoritma pencarian nilai minimum pada selection sort!

Jawaban

```
void SelectionSort(){
    for(int i=0; i<jumData-1; i++){
        int min=i;
        for (int j=i+1; j<jumData; j++){
            if(data[j]<data[min]){
                min=j;
            }
        }
        int temp=data[i];
        data[i]=data[min];
        data[min]=temp;
    }
}
```

3. Pada Insertion sort , jelaskan maksud dari kondisi pada perulangan

```
while (j>=0 && data[j]>temp)
```

Jawaban : kode diatas merupakan bagian dari insertion sort yang bertugas untuk mencari posisi yang tepat bagi angka dengan cara menggeser angka-angka yang besar darinya.

4. Pada Insertion sort, apakah tujuan dari perintah

```
data[j+1]= data[j];
```

Jawaban: tujuannya untuk memberikan ruang bagi nilai temp agar bisa disisipkan di posisi yang tepat, karena pada insertion membandingkan nilai dengan elemen sebelah kirinya yang sudah terurut.

5.3 PRAKTIKUM 2 – Sorting (menggunakan array of object)

5.3.1 Langkah Praktikum 2 – Mengurutkan data mahasiswa berdasarkan IPK (bubble sort)

5.3.2 Langkah – langkah praktikum 2

1. Buatlah class dengan nama Mahasiswa.

2. Untuk lebih jelasnya class tersebut dapat dilihat pada potongan kode di bawah ini

Jawaban :

```java
public class Mahasiswa13 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    //Konstruktor default
    Mahasiswa13() {
    }

    //Konstruktor berparameter
    public Mahasiswa13(String nm, String name, String kls, double ip) {
        nim = nm;
        nama = name;
        ipk = ip;
        kelas = kls;
    }

    void tampilInformasi(){
        System.out.println("Nama: " + nama);
        System.out.println("NIM: " + nim);
        System.out.println("Kelas: " + kelas);
        System.out.println("IPK: " + ipk);
    }


}
```

3. Buat class MahasiswaBerprestasi seperti di bawah ini!

Jawaban :

```
public class MahasiswaBerprestasi13 {
    Mahasiswa13 [] listMhs= new Mahasiswa13[5];
    int idx;
}
```

4. Tambahkan method tambah() di dalam class tersebut! Method tambah() digunakan untuk menambahkan objek dari class Mahasiswa ke dalam atribut listMhs.

Jawaban :

```
public class MahasiswaBerprestasi13 {
    Mahasiswa13 [] listMhs= new Mahasiswa13[5];
    int idx;

    void tambah (Mahasiswa13 m){
        if (idx<listMhs.length){
            listMhs[idx]=m;
            idx++;
        }else {
            System.out.println("data sudah penuh");
        }
    }
```

5. Tambahkan method tampil() di dalam class tersebut! Method tampil() digunakan untuk menampilkan semua data mahasiswa-mahasiswa yang ada di dalam class tersebut! Perhatikan penggunaan sintaks for yang agak berbeda dengan for yang telah dipelajari sebelumnya, meskipun secara konsep sebenarnya mirip.

Jawaban:

```
void tampil (){
    for (Mahasiswa13 m:listMhs){
        m.tampilInformasi();
        System.out.println("--------------------------------");
    }
}
```

6. Tambahkan method bubbleSort() di dalam class tersebut!

Jawaban:

```
void bubbleSort(){
    for (int i=0; i<listMhs.length-i; i++){
        for (int j=1; j<listMhs.length; j++){
            if (listMhs[j].ipk>listMhs[j-1].ipk){
                Mahasiswa13 tmp = listMhs[j];
                listMhs[j]=listMhs[j-1];
                listMhs[j-1]=tmp;
            }
        }
    }
}
```

7. Buat class MahasiswaDemo, kemudian buatlah sebuah objek MahasiswaBerprestasi dan buatlah 5 objek mahasiswa kemudian tambahkan semua objek mahasiswa tersebut dengan memanggil fungsi tambah pada objek MahasiswaBerprestasi. Silakan dipanggil fungsi tampil() untuk melihat semua data yang telah dimasukan, urutkan data tersebut dengan memanggil fungsi bubbleSort() dan yang terakhir panggil fungsi tampil kembali.

Jawaban:

```
public class MahasiswaDemo13 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        MahasiswaBerprestasi13 list = new MahasiswaBerprestasi13();
        Mahasiswa13 m1 = new Mahasiswa13(nm: "123", name: "Zidan", kls: "2A", ip: 3.2);
        Mahasiswa13 m2 = new Mahasiswa13(nm: "124", name: "Ayu", kls: "2A", ip: 3.1);
        Mahasiswa13 m3 = new Mahasiswa13(nm: "125", name: "Sofi", kls: "2A", ip: 3.9);
        Mahasiswa13 m4 = new Mahasiswa13(nm: "126", name: "Sita", kls: "2A", ip: 3.9);
        Mahasiswa13 m5 = new Mahasiswa13(nm: "127", name: "Miki", kls: "2A", ip: 3.7);

        list.tambah(m1);
        list.tambah(m2);
        list.tambah(m3);
        list.tambah(m4);
        list.tambah(m5);

        System.out.println(x: "Data Mahasiswa sebelum sorting: ");
        list.tampil();

        System.out.println(x: "Data Mahasiswa setelah sorting berdasarkan IPK (DES)");
        list.bubbleSort();
        list.tampil();

    }
```

5.3.3 Verifikasi Hasil Percobaan

Data Mahasiswa sebelum di sorting:

```
Data Mahasiswa sebelum sorting:
Nama: Zidan
NIM: 123
Kelas: 2A
IPK: 3.2
---------------------------------
Nama: Ayu
NIM: 124
Kelas: 2A
IPK: 3.5
---------------------------------
Nama: Sofi
NIM: 125
Kelas: 2A
IPK: 3.1
---------------------------------
Nama: Sita
NIM: 126
Kelas: 2A
IPK: 3.9
---------------------------------
Nama: Miki
NIM: 127
Kelas: 2A
IPK: 3.7
```

Data Mahasiswa setelah di sorting

```
---------------------------------
Data yang sudah terurut menggunakan sorting berdasarkan IPK (DES)
Nama: Sita
NIM: 126
Kelas: 2A
IPK: 3.9
---------------------------------
Nama: Miki
NIM: 127
Kelas: 2A
IPK: 3.7
---------------------------------
Nama: Ayu
NIM: 124
Kelas: 2A
IPK: 3.5
---------------------------------
Nama: Zidan
NIM: 123
Kelas: 2A
IPK: 3.2
---------------------------------
Nama: Sofi
NIM: 125
Kelas: 2A
IPK: 3.1
---------------------------------
```

5.3.4 Pertanyaan

1. Perhatikan perulangan di dalam bubbleSort() di bawah ini:

```
for (int i=0; i<listMhs.length-1; i++){
    for (int j=1; j<listMhs.length-i; j++){
```

a.Mengapa syarat dari perulangan i adalah i<listMhs.length – 1?

Jawaban: Syarat ini menentukan jumlah pass atau putaran besar. Jika memiliki n data, hanya butuh n-1 putaran untuk memastikan semua data teurut.

b.Mengapa syarat dari perulangan j adalalh j<listMhs.length-i ?

Jawaban: karena setiap putaran i selesai, satu data terbesar (atau terkecil) akan "tenggelam" ke posisi paling ujung. Dengan menggunakan -i,tidak perlu lagi memeriksa data di ujung yang sudah pasti benar posisinya, sehingga proses perulangan menjadi lebih cepat.

c. Jika banyak data di dalam listMhs adalah 50, maka berapakali perulangan i akan berlangsung? Dan ada berapa Tahap bubble sort yang ditempuh?

Jawaban: Perulangan i: Berlangsung sebanyak 49 kali, Ada 49 tahapan (pass). Dalam setiap tahap, perulangan j akan melakukan perbandingan yang jumlahnya semakin berkurang seiring bertambahnya i.

2. Modifikasi program diatas dimana data mahasiswa bersifat dinamis (input dari keyborad) yang terdiri dari nim, nama, kelas, dan ipk!

Jawaban:

```
Run | Debug | Run main | Debug main
public static void main(String[] args) {
     Scanner sc = new Scanner(System.in);
    MahasiswaBerprestasi13 list = new MahasiswaBerprestasi13();

    System.out.print(s: "Masukkan jumlah mahasiswa: ");
    int jml = sc.nextInt();
    sc.nextLine();

    for (int i = 0; i < jml; i++) {
        System.out.println("--- Data Mahasiswa ke-" + (i + 1) + " ---");
        System.out.print(s: "NIM  : ");
        String nim = sc.nextLine();
        System.out.print(s: "Nama   : ");
        String nama = sc.nextLine();
        System.out.print(s: "Kelas : ");
        String kelas = sc.nextLine();
        System.out.print(s: "IPK   : ");
        double ipk = sc.nextDouble();
        sc.nextLine();

        Mahasiswa13 m = new Mahasiswa13(nim, nama, kelas, ipk);
        list.tambah(m);
        System.out.println();
    }
```

5.3.5 Mengurutkan Data Mahasiswa Berdasarkan IPK (Selection Sort)

5.3.1. Langkah-langkah Percobaan.

1. Lihat kembali class MahasiswaBerprestasi, dan tambahkan method selectionSort() di dalamnya! Method ini juga akan melakukan proses sorting secara ascending, tetapi menggunakan pendekatan selection sort.

Jawaban:

```
void SelectionSort() {
for (int i = 0; i < listMhs.length - 1; i++) {
    int idxMin = i;
    for (int j = i + 1; j < listMhs.length; j++) {
        // Mencari IPK yang lebih kecil
        if (listMhs[j].ipk < listMhs[idxMin].ipk) {
            idxMin = j;
        }
    }
    Mahasiswa13 tmp = listMhs[idxMin];
    listMhs[idxMin] = listMhs[i];
    listMhs[i] = tmp;
}
```

2. Setelah itu, buka kembali class MahasiswaDemo, dan di dalam method main() tambahkan baris program untuk memanggil method selectionSort() tersebut, kemudian panggil method tampil() untuk menampilkan data yang sudah diurutkan!

Jawaban:

```
System.out.println(x: "Data yang sudah terurut menggunakan SELECTION SORT(ASC)");
list.SelectionSort();
list.tampil();
```

3. Coba jalankan kembali class MahasiswaDemo, dan amati hasilnya! Apakah kini data mahasiswa telah tampil urut menaik berdasar ipk?

Jawaban:

```
Data yang sudah terurut menggunakan SELECTION SORT(ASC)
Nama: ila
NIM: 124
Kelas: 2B
IPK: 3.1
--------------------------------
Nama: udin
NIM: 127
Kelas: 2B
IPK: 3.2
--------------------------------
Nama: tika
NIM: 126
Kelas: 2B
IPK: 3.3
--------------------------------
Nama: Agus
NIM: 125
Kelas: 2B
IPK: 3.6
--------------------------------
Nama: Ali
NIM: 123
Kelas: 2B
IPK: 3.9
--------------------------------
```

5.3.7 Pertanyaan

Di dalam method selection sort, terdapat baris program seperti di bawah ini:

```
int idxMin=i;
for (int j=i+1; j<listMhs.length; j++){
    if (listMhs[j].ipk<listMhs[idxMin].ipk){
        idxMin=j;
    }
}
```

Untuk apakah proses tersebut, jelaskan!

Jawaban: Tujuan utama dari kode tersebut adalah menyisir sekumpulan data untuk menemukan mana data yang nilainya paling kecil di antara data-data yang

belum terurut. Setelah perulangan j selesai, variabel idxMin akan berisi alamat atau indeks dari mahasiswa yang memiliki IPK paling kecil.

Dalam konteks Selection Sort secara utuh, setelah proses pencarian ini selesai, barulah dilakukan Swap (penukaran) antara listMhs[i] dengan listMhs[idxMin].

5.4.1 Langkah-langkah Percobaan

1. Lihat kembali class MahasiswaBerprestasi, dan tambahkan method insertionSort() di dalamnya. Method ini juga akan melakukan proses sorting secara ascending, tetapi menggunakan pendekatan Insertion Sort.

Jawaban:

```
void InsertionSort(){
  for (int i=1; i<listMhs.length; i++){
      Mahasiswa13 temp = listMhs[i];
      int j=i;
      while (j>0 && listMhs[j-1].ipk>temp.ipk){
          listMhs[j]=listMhs[j-1];
          j--;
      }
      listMhs[j]=temp;
  }
}
```

2. Setelah itu, buka kembali class MahasiswaDemo, dan di dalam method main() tambahkan baris program untuk memanggil method insertionSort() dan tampil () tersebut!

Jawaban:

```
System.out.println(x: "Data yang sudah terurut menggunakan INSERTION SORT(ASC)");
list.InsertionSort();
list.tampil();
```

3. Coba jalankan kembali class MahasiswaDemo, dan amati hasilnya! Apakah kini data mahasiswa telah tampil urut menaik berdasar ipk?

Jawaban:

```
Data yang sudah terurut menggunakan INSERTION SORT(ASC)
Nama: dika
NIM: 222
Kelas: 2c
IPK: 3.0
---------------------------------
Nama: susi
NIM: 444
Kelas: 2c
IPK: 3.1
---------------------------------
Nama: yayuk
NIM: 555
Kelas: 2c
IPK: 3.4
---------------------------------
Nama: ayu
NIM: 111
Kelas: 2c
IPK: 3.7
---------------------------------
Nama: ila
NIM: 333
Kelas: 2c
IPK: 3.8
```

5.4.3 Pertanyaan Ubahlah fungsi pada InsertionSort sehingga fungsi ini dapat melaksanakan proses sorting dengan cara descending.

Jawaban:

```
void insertionSortDescending() {
for (int i = 1; i < listMhs.length; i++) {
    Mahasiswa13 temp = listMhs[i];
    int j = i;
    while (j > 0 && listMhs[j - 1].ipk < temp.ipk) {
        listMhs[j] = listMhs[j - 1];
        j--;
    }
    listMhs[j] = temp;
}
```

5.5 Latihan Praktikum

1. Tambah data digunakan untuk menambahkan data dosen

Jawaban:

```
public class Dosen13 {
    String kode;
    String nama;
    boolean jenisKelamin;
    int usia;

public Dosen13(String kd, String name, boolean jk, int age){
    this.kode = kd;
    this.nama = name;
    this.jenisKelamin = jk;
    this.usia = age;
}
```

2. Tampil data digunakan untuk menampilkan data seluruh dosen

Jawaban:

```
public void tampil() {
    String jk = (jenisKelamin) ? "Pria" : "Wanita";
    System.out.printf(format: "Kode: %s | Nama: %-15s | JK: %-7s | Usia: %d\n",
    }
}
```

3. Sorting ASC digunakan untuk mengurutkan data dosen berdasarkan usia dimulai dari dosen termuda ke dosen tertua menggunakan bublle Sort.

Jawaban:

```
}
public void sortingASC() {
    for (int i = 0; i < idx - 1; i++) {
        for (int j = 0; j < idx - i - 1; j++) {
            if (dataDosen[j].usia > dataDosen[j + 1].usia) {
                Dosen13 temp = dataDosen[j];
                dataDosen[j] = dataDosen[j + 1];
                dataDosen[j + 1] = temp;
            }
        }
    }
}
```

4.Sorting DSC digunakan untuk mengurutkan data dosen berdasarkan usia dimulai dari tertua ke dosen termuda dapat menggunakan algoritma selection sort atau insertion sort.

Jawaban:

```
}
public void sortingDSC() {
    for (int i = 0; i < idx - 1; i++) {
        int maxIdx = i;
        for (int j = i + 1; j < idx; j++) {
            if (dataDosen[j].usia > dataDosen[maxIdx].usia) {
                maxIdx = j;
            }
        }
        Dosen13 temp = dataDosen[maxIdx];
        dataDosen[maxIdx] = dataDosen[i];
        dataDosen[i] = temp;
    }
}
```